ALPE FORMACIÓN · SANTANDER

Edición 2026 · PHP 8.x

CURSO DE PHP · CLASE 36

Pruebas

- Por qué hacer pruebas
- PHPUnit y php artisan test
- La anatomía de una prueba
- Pruebas unitarias
- La base de datos de pruebas
- Factories en las pruebas
- Pruebas de funcionalidad
- Probar la API
- Las pruebas encuentran errores



Ramón Abramo

+25 años de experiencia docente · Santander, Cantabria

ALPE

FORMACIÓN PROFESIONAL

🧭 Contenido de la clase

¿Cómo sabéis que la academia funciona? Hasta ahora, abriendo el navegador y probando a mano: entrar con Marta, crear un curso, salir, entrar con Pablo, comprobar que no puede... Ahora contad cuántas cosas hemos construido. ¿De verdad vais a probarlas **todas** cada vez que cambiéis una línea?

Las **pruebas automáticas** son programas que prueban nuestro programa. Se escriben una vez y se ejecutan siempre que queráis, en menos de un segundo. Y os aviso de algo que os va a pasar hoy mismo: en cuanto escribamos unas pocas, **van a encontrar un error** que llevaba varias clases escondido. 🕵️

•

👌 Cómo vamos a trabajar



1

📘 Por qué hacer pruebas



2

👍 PHPUnit y php artisan test



3

🔍 La anatomía de una prueba



4

🚀 Pruebas unitarias



5

🤔 La base de datos de pruebas



6

😮 Factories en las pruebas



7

📘 Pruebas de funcionalidad



8

👌 Probar la API



9

⚠️ Las pruebas encuentran errores



10

🚀 Ejercicios



11

👋 Lo que tenéis que llevaros de hoy





CLASE 36

👌 Cómo vamos a trabajar

Borrad las dos pruebas de ejemplo que trae Laravel (`tests/Feature/ExampleTest.php` y `tests/Unit/ExampleTest.php`) y copiad el contenido de `ejemplos/` encima del proyecto:

Esquema

```
tests/Unit/CalculadoraPreciosTest.php   ← NUEVA: prueba unitaria del servicio de la clase 34
tests/Feature/CursoTest.php             ← NUEVA: el CRUD de cursos con sus permisos
tests/Feature/CursoModeloTest.php       ← NUEVA: el modelo Curso
tests/Feature/AutenticacionTest.php     ← NUEVA: login, registro y logout
tests/Feature/Api/CursoApiTest.php      ← NUEVA: la API de la clase 35
database/factories/UserFactory.php      ← estados admin() y editor()
database/factories/CursoFactory.php     ← corregida (ya veréis por qué en el punto 9)
database/seeders/CursoSeeder.php        ← corregido
```

Las pruebas usan **SQLite** en memoria, así que la extensión `pdo_sqlite` tiene que estar activada. En Laragon: menú **PHP → Extensiones → pdo_sqlite**.



CLASE 36 · PUNTO 1

📘 Por qué hacer pruebas

Sin pruebas	Con pruebas
Cambiáis algo y cruzáis los dedos 🤞	Cambiáis algo y ejecutáis `php artisan test`
Los errores los encuentra el cliente	Los errores los encuentra la prueba, en vuestro ordenador
Da miedo mejorar el código «por si se rompe algo»	Podéis reorganizar el código con tranquilidad
¿Cómo funcionaba esto? A leer el código	La prueba **documenta** lo que debe hacer

Hay dos grandes tipos de pruebas [1]:

	Unitarias (`tests/Unit`)	De funcionalidad (`tests/Feature`)
Qué prueban	Una clase suelta	Un recorrido completo: petición → rutas → middleware → controlador → base de datos → respuesta
¿Arrancan Laravel?	No	Sí
Velocidad	Rapidísimas	Rápidas
Ejemplo	«La calculadora suma bien el IVA»	«Un editor puede crear un curso, pero un usuario normal recibe un 403»



CLASE 36 · PUNTO 2

👍 PHPUnit y php artisan test

Laravel trae instalado **PHPUnit**, la herramienta de pruebas estándar de PHP, configurado en `phpunit.xml`. Para ejecutar todas las pruebas:

terminal — php-paso-a-paso/clase36 — Visual Studio Code

terminal

```
php artisan test
```

Esquema

```
   PASS  Tests\Unit\CalculadoraPreciosTest
  ✓ suma el iva
  ✓ redondea a dos decimales
  ✓ aplica el descuento
  ✓ no permite mas descuento del maximo
  ✓ formatea como en españa

   PASS  Tests\Feature\CursoTest
  ✓ el listado muestra solo los cursos publicados                    0.02s
  ✓ la ficha de un curso que no existe no se encuentra               0.01s
  ✓ un invitado no puede crear cursos                                0.02s
  ✓ un usuario normal no tiene permiso                               0.04s
  ✓ un editor puede crear un curso                                   0.01s
  ...

  Tests:    24 passed (77 assertions)
  Duration: 0.75s
```

24 comprobaciones de toda la aplicación en **menos de un segundo**. Ninguna persona prueba tan rápido. 😎

Comando	Qué hace
`php artisan test`	Todas las pruebas
`php artisan test --filter=CursoTest`	Solo las de esa clase (o las que contengan ese texto)
`php artisan test tests/Unit`	Solo una carpeta
`php artisan test --stop-on-failure`	Se para en el primer fallo
`php artisan make:test CursoTest`	Crea una prueba de funcionalidad
`php artisan make:test CalculadoraTest --unit`	Crea una prueba unitaria



💡 Laravel también ofrece **Pest**, otra herramienta de pruebas con una sintaxis más corta (`it('suma el iva', function () { ... })`). Por debajo usa PHPUnit, y todo lo que aprendáis hoy os sirve igual.



CLASE 36 · PUNTO 3

🔍 La anatomía de una prueba

Una prueba es un **método** dentro de una clase que hereda de `TestCase`. Cada método prueba **una sola cosa**, y su nombre lo dice en lenguaje normal:

ejemplo.php — php-paso-a-paso/clase36 — Visual Studio Code

ejemplo.php

```php
#[Test]
public function elListadoMuestraSoloLosCursosPublicados(): void
{
    // Preparar
    Curso::factory()->create(['titulo' => 'Python básico', 'publicado' => true]);
    Curso::factory()->sinPublicar()->create(['titulo' => 'Linux avanzado']);

    // Actuar
    $respuesta = $this->get('/cursos');

    // Comprobar
    $respuesta->assertOk();
    $respuesta->assertSee('Python básico');
    $respuesta->assertDontSee('Linux avanzado');
}
```

Casi todas las pruebas siguen el patrón de las **tres A** (*Arrange, Act, Assert*):

1. **Preparar**: crear los datos que necesita la prueba.

2. **Actuar**: hacer lo que queremos probar.
3. **Comprobar**: las ***aserciones*** (`assert...`), que fallan si el resultado no es el esperado.

El atributo `#[Test]` le dice a PHPUnit que ese método es una prueba. La otra forma, que veréis mucho, es empezar el nombre por `test`: `public function test_el_listado_muestra_solo_los_publicados()`.



💡 Si una prueba falla, **el nombre** es lo primero que leeréis. «un editor no puede borrar pero un administrador sí» os dice qué se ha roto sin abrir el código.



CLASE 36 · PUNTO 4

🚀 Pruebas unitarias

El servicio `CalculadoraPrecios` de la clase 34 es perfecto para empezar: una clase normal, sin base de datos (`CalculadoraPreciosTest.php`):

ejemplo.php — php-paso-a-paso/clase36 — Visual Studio Code

ejemplo.php

```php
namespace Tests\Unit;

use App\Servicios\CalculadoraPrecios;
use PHPUnit\Framework\Attributes\Test;
use PHPUnit\Framework\TestCase;

// Prueba unitaria: una clase sola, sin Laravel, sin base de datos. Muy rápida.
class CalculadoraPreciosTest extends TestCase
{
    private CalculadoraPrecios $calculadora;

    // Se ejecuta antes de CADA prueba: cada una empieza con una calculadora nueva
    protected function setUp(): void
    {
        parent::setUp();

        $this->calculadora = new CalculadoraPrecios(iva: 0.21, descuentoMaximo: 30);
    }

    #[Test]
    public function sumaElIva(): void
    {
        $this->assertSame(121.0, $this->calculadora->conIva(100));
    }

    #[Test]
    public function noPermiteMasDescuentoDelMaximo(): void
    {
        // Piden un 80 %, pero el máximo es un 30 %
        $this->assertSame(70.0, $this->calculadora->conDescuento(100, 80));
    }

    #[Test]
    public function formateaComoEnEspaña(): void
    {
        $this->assertSame('1.234,50 €', $this->calculadora->formatear(1234.5));
    }
}
```

¿Recordáis que en la clase 34 os dije que la calculadora recibía el IVA en el constructor «para poder probarla»? Aquí lo tenéis: creamos una calculadora con los valores que queremos, sin depender de la configuración.

Aserción	Comprueba que...
`assertSame($esperado, $real)`	Son idénticos (valor **y** tipo: `121.0` no es `121`)
`assertEquals($esperado, $real)`	Son iguales (sin mirar el tipo)
`assertTrue($x)` / `assertFalse($x)`	Es `true` / `false`
`assertNull($x)` / `assertNotNull($x)`	Es / no es `null`
`assertCount(3, $lista)`	La lista tiene 3 elementos
`assertStringContainsString('Ana', $texto)`	El texto contiene «Ana»

Fijaos en la importancia de **probar los límites**: no basta con un descuento normal, hay que probar qué pasa cuando alguien pide más del máximo. Ahí es donde se esconden los errores.



CLASE 36 · PUNTO 5

🤔 La base de datos de pruebas

Las pruebas de funcionalidad crean usuarios, cursos, matrículas... ¿Y si borran los datos de nuestra base de datos de desarrollo? Por eso **nunca** se usa la base de datos de verdad. Mirad estas líneas de `phpunit.xml`:

plantilla.php — php-paso-a-paso/clase36 — Visual Studio Code

plantilla.php

```
<env name="APP_ENV" value="testing"/>
<env name="DB_CONNECTION" value="sqlite"/>
<env name="DB_DATABASE" value=":memory:"/>
```

Durante las pruebas, Laravel usa una base de datos **SQLite en memoria**: se crea al empezar, vive solo en la RAM y desaparece al terminar. Rapidísima y sin riesgo [2].

Cada clase de pruebas que use la base de datos lleva el *trait* `RefreshDatabase`:

ejemplo.php — php-paso-a-paso/clase36 — Visual Studio Code

ejemplo.php

```
class CursoTest extends TestCase
{
    // Cada prueba empieza con la base de datos de pruebas vacía y recién migrada
    use RefreshDatabase;
```

Ejecuta las migraciones y, después de **cada** prueba, deshace los cambios. Así cada prueba empieza de cero y no depende de lo que hicieron las demás: el orden en que se ejecuten da igual.



CLASE 36 · PUNTO 6

😮 Factories en las pruebas

Las *factories* de la clase 30 brillan de verdad aquí: cada prueba crea **exactamente** los datos que necesita, en una línea. Para no repetir el rol a mano, añadimos dos **estados** a `UserFactory`:

ejemplo.php — php-paso-a-paso/clase36 — Visual Studio Code

ejemplo.php

```php
// Estados para las pruebas: User::factory()->admin()->create()
public function admin(): static
{
    return $this->state(fn () => ['rol' => Rol::Admin]);
}

public function editor(): static
{
    return $this->state(fn () => ['rol' => Rol::Editor]);
}
```

ejemplo.php — php-paso-a-paso/clase36 — Visual Studio Code

ejemplo.php

```php
$editora = User::factory()->editor()->create();
$curso = Curso::factory()->sinPublicar()->create(['titulo' => 'Linux avanzado']);
$curso = Curso::factory()->for($categoria)->create();     // con una categoría concreta
$curso = Curso::factory()->make(['precio' => 100]);       // sin guardar: aún más rápido
```



💡 Poned en la prueba **solo los datos que importan** para esa prueba. Si la prueba va de que los no publicados no salen, escribid `'publicado' => false` aunque la *factory* ya lo haga a veces: así quien lea la prueba entiende por qué funciona.



CLASE 36 · PUNTO 7

📘 Pruebas de funcionalidad

En una prueba de funcionalidad hacemos **peticiones HTTP** a la aplicación entera, sin servidor ni navegador [3]. `CursoTest.php`:

ejemplo.php — php-paso-a-paso/clase36 — Visual Studio Code

ejemplo.php

```php
#[Test]
public function unInvitadoNoPuedeCrearCursos(): void
{
    $this->get('/cursos/create')->assertRedirect('/login');
    $this->post('/cursos', $this->datosValidos())->assertRedirect('/login');

    $this->assertDatabaseCount('cursos', 0);
}

#[Test]
public function unUsuarioNormalNoTienePermiso(): void
{
    $usuario = User::factory()->create();

    $this->actingAs($usuario)
        ->post('/cursos', $this->datosValidos())
        ->assertForbidden();
}

#[Test]
public function unEditorPuedeCrearUnCurso(): void
{
    $editora = User::factory()->editor()->create();

    $respuesta = $this->actingAs($editora)->post('/cursos', $this->datosValidos());

    $curso = Curso::firstOrFail();
    $respuesta->assertRedirect(route('cursos.show', $curso));
    $respuesta->assertSessionHas('exito');
    $this->assertDatabaseHas('cursos', [
        'titulo' => 'Vue.js desde cero',
        'slug' => 'vuejs-desde-cero',
    ]);
}
```

- `actingAs($usuario)`: hace la petición como si ese usuario hubiera iniciado sesión. No hace falta pasar por el formulario de login.
- Las pruebas no necesitan el token CSRF: Laravel lo desactiva automáticamente al probar.
- `datosValidos()` es un método auxiliar de la propia clase que devuelve un curso correcto; cada prueba cambia solo lo que le interesa.

Las aserciones de las respuestas

Aserción	Comprueba
`assertOk()` / `assertStatus(200)`	Código 200
`assertNotFound()` / `assertForbidden()`	Código 404 / 403
`assertRedirect('/login')`	Que redirige a esa dirección
`assertSee('texto')` / `assertDontSee('texto')`	Que el HTML contiene / no contiene el texto
`assertSessionHas('exito')`	Que se guardó ese mensaje *flash*
`assertSessionHasErrors(['titulo'])`	Que la validación falló en esos campos
`assertViewIs('cursos.index')`	Que se pintó esa vista

Las aserciones de la base de datos y del usuario

Aserción	Comprueba
`assertDatabaseHas('cursos', [...])`	Que existe una fila con esos datos
`assertDatabaseMissing('cursos', [...])`	Que no existe
`assertDatabaseCount('cursos', 0)`	Cuántas filas hay
`assertSoftDeleted($curso)`	Que se borró con borrado lógico
`assertAuthenticatedAs($usuario)` / `assertGuest()`	Quién tiene la sesión iniciada / nadie

Falsificar servicios: subir archivos sin archivos

¿Cómo probamos la subida de la imagen sin llenar `storage/` de basura? Con los ***fakes*** («falsificaciones») de Laravel:

ejemplo.php — php-paso-a-paso/clase36 — Visual Studio Code

ejemplo.php

```php
#[Test]
public function seGuardaLaImagenDelCurso(): void
{
    // Un disco falso: no se escribe nada en storage/ de verdad
    Storage::fake('public');
    $editora = User::factory()->editor()->create();

    $this->actingAs($editora)->post('/cursos', $this->datosValidos([
        'imagen' => UploadedFile::fake()->image('portada.jpg', 800, 500),
    ]));

    $curso = Curso::firstOrFail();
    $this->assertNotNull($curso->imagen);
    Storage::disk('public')->assertExists($curso->imagen);
}
```

Hay *fakes* para casi todo: `Mail::fake()`, `Notification::fake()`, `Queue::fake()`, `Event::fake()`, `Http::fake()`... Los usaremos en la clase 37.

La prueba de la autenticación (`AutenticacionTest.php`) comprueba el login, el registro y el cierre de sesión:

ejemplo.php — php-paso-a-paso/clase36 — Visual Studio Code

ejemplo.php

```php
#[Test]
public function alRegistrarseSeCifraLaContraseñaYSeIniciaLaSesion(): void
{
    $this->post('/registro', [
        'name' => 'Lucía',
        'email' => 'lucia@example.com',
        'password' => 'secreta123',
        'password_confirmation' => 'secreta123',
    ])->assertRedirect('/');

    $usuario = User::where('email', 'lucia@example.com')->firstOrFail();
    $this->assertNotSame('secreta123', $usuario->password);
    $this->assertAuthenticatedAs($usuario);
}
```



CLASE 36 · PUNTO 8

👌 Probar la API

Para la API usamos las versiones JSON de las peticiones (`getJson`, `postJson`, `deleteJson`...), que ya envían la cabecera `Accept: application/json`, y `Sanctum::actingAs()` para simular un token

(`CursoApiTest.php`):

ejemplo.php — php-paso-a-paso/clase36 — Visual Studio Code

ejemplo.php

```php
#[Test]
public function elListadoDevuelveLosCursosPaginados(): void
{
    Curso::factory(3)->create(['publicado' => true]);

    $this->getJson('/api/v1/cursos?por_pagina=2')
        ->assertOk()
        ->assertJsonCount(2, 'data')
        ->assertJsonPath('meta.total', 3)
        ->assertJsonStructure([
            'data' => [['id', 'titulo', 'precio', 'categoria' => ['nombre']]],
            'links',
            'meta',
        ]);
}

#[Test]
public function soloElAdministradorPuedeBorrar(): void
{
    $curso = Curso::factory()->create();

    Sanctum::actingAs(User::factory()->editor()->create());
    $this->deleteJson("/api/v1/cursos/{$curso->id}")->assertForbidden();

    Sanctum::actingAs(User::factory()->admin()->create());
    $this->deleteJson("/api/v1/cursos/{$curso->id}")->assertNoContent();

    $this->assertSoftDeleted($curso);
}
```

Aserción	Comprueba
`assertJson(['mensaje' => '...'])`	Que el JSON contiene esos datos
`assertJsonPath('data.titulo', 'Vue.js desde cero')`	Un valor concreto, con notación de puntos
`assertJsonCount(2, 'data')`	Cuántos elementos tiene una lista
`assertJsonStructure([...])`	Que existen esas claves (sin mirar los valores)
`assertJsonValidationErrors(['titulo'])`	Que hay errores de validación en esos campos
`assertCreated()`, `assertNoContent()`, `assertUnauthorized()`, `assertUnprocessable()`	201, 204, 401 y 422

Y fijaos en lo que **no** hemos tenido que hacer: ni pedir tokens, ni copiarlos, ni esperar al `throttle`. Probar la API a mano en la clase 35 nos llevó un buen rato; estas pruebas tardan una décima de segundo.



CLASE 36 · PUNTO 9

⚠️ Las pruebas encuentran errores

Os lo prometí al principio. Al escribir `CursoModeloTest.php`, una prueba tan inocente como esta **falló**:

ejemplo.php — php-paso-a-paso/clase36 — Visual Studio Code

ejemplo.php

```php
#[Test]
public function alGuardarElTituloSeGeneraElSlug(): void
{
    $curso = Curso::factory()->create(['titulo' => '  Diseño web con Figma ']);

    $this->assertSame('Diseño web con Figma', $curso->titulo);
    $this->assertSame('diseno-web-con-figma', $curso->slug);
}
```

Esquema

```
 FAIL  Tests\Feature\CursoModeloTest
× al guardar el titulo se genera el slug
Failed asserting that two strings are identical.
-'diseno-web-con-figma'
+'git-intermedio'
```

¿Qué pasaba? En la clase 30, antes de tener el *mutator* del título, la `CursoFactory` generaba su propio `slug` a partir de **su** título inventado. En la clase 31 añadimos el *mutator*, que genera el `slug` al asignar el título… pero la *factory* seguía enviando el suyo **después**, y lo pisaba. Si le pasábamos otro título, el curso se quedaba con el título nuevo y el `slug` del inventado.

El arreglo es una línea: quitar el `slug` de la *factory* (y del *seeder*), porque ahora es cosa del modelo:

ejemplo.php — php-paso-a-paso/clase36 — Visual Studio Code

ejemplo.php

```php
return [
    // Si no le decimos la categoría, crea una nueva
    'categoria_id' => Categoria::factory(),
    // El slug lo genera el mutator del modelo a partir del título (clase 31)
    'titulo' => $titulo,
    ...
];
```

Este error llevaba **cinco clases** escondido, y ninguno de nosotros lo vio probando a mano. Una prueba de cinco líneas lo encontró en una décima de segundo. Y a partir de ahora, si alguien lo vuelve a introducir, la prueba avisará. Esa es la idea: **cada error que arregláis, una prueba que impide que vuelva**.



💡 Hay quien escribe la prueba **antes** que el código: primero la prueba (que falla), después el código justo para que pase, y después se mejora. Se llama **TDD** (*Test-Driven Development*) y es una forma de trabajar muy extendida.



CLASE 36 · PUNTO 10

🚀 Ejercicios

Las soluciones se copian dentro del proyecto, respetando sus carpetas.

- ☐ **01 · Más pruebas del modelo.** Añadid a `CursoModeloTest` una prueba de que el *scope* `publicados()` descarta los borradores y otra de que un curso borrado no aparece en `Curso::count()` pero sí en `Curso::withTrashed()->count()`. → `CursoModeloTest.php`
- ☐ **02 · El formulario de contacto.** Probad que un mensaje correcto llama al `Notificador` (clase 34) con el correo de la academia, y que uno incompleto no lo llama. Para no escribir en el log, sustituid el notificador en el contenedor por uno falso: `$this->app->instance(Notificador::class, $falso)`, donde `$falso` es una clase anónima que implementa el contrato y guarda lo que recibe. → `ContactoTest.php`
- ☐ **03 · Probar los comandos.** Probad el comando `academia:cursos` (con una categoría sin cursos y con una que tiene uno) y `academia:estadisticas`. Pista: `$this->artisan('academia:cursos', ['--categoria' => 'diseno'])->expectsOutputToContain('...')->assertSuccessful()`. → `ComandosTest.php`



🏆 **Reto:** activad Xdebug en Laragon y ejecutad `php artisan test --coverage`. Os dirá qué porcentaje de cada fichero se ejecuta durante las pruebas. Buscad el controlador con menos cobertura y escribid las pruebas que le faltan.



CLASE 36 · PUNTO 11

👋 Lo que tenéis que llevaros de hoy

Idea	En una frase
Pruebas automáticas	Programas que prueban nuestro programa en un segundo
Unitarias / de funcionalidad	Una clase suelta / un recorrido HTTP completo
`php artisan test`	Ejecuta todas; `--filter` para unas pocas
Las tres A	Preparar, actuar, comprobar
`#[Test]`	Marca un método como prueba; el nombre dice lo que prueba
`RefreshDatabase` + SQLite en memoria	Cada prueba con su base de datos limpia, sin tocar la de verdad
Factories y estados	Los datos justos de cada prueba en una línea
`actingAs()` / `Sanctum::actingAs()`	Probar como un usuario / con un token
`assertDatabaseHas`, `assertSessionHasErrors`, `assertJsonPath`…	Las comprobaciones más útiles
Fakes	`Storage::fake()`, `Mail::fake()`…: probar sin efectos reales
Cada error, una prueba	Para que no vuelva

📚 Fuentes y documentación oficial

1. Laravel: Testing, Getting Started
2. Laravel: Database Testing
3. Laravel: HTTP Tests
4. Laravel: Sanctum, Testing
5. Laravel: Console Tests
6. PHPUnit: documentación